```
from google.colab import drive
# Tenta desmontar o Drive para garantir uma nova conexão limpa
try:
    drive.flush_and_unmount()
except:
    pass
# Monta o Drive novamente. Você deve ver um link e pedir a permissão.
drive.mount('/content/drive')
```

```
Drive not mounted, so nothing to flush and unmount.
Mounted at /content/drive
```

Comece a programar ou gere código com IA.

```
CAMINHO_DO_SEU_ARQUIVO = '/content/drive/MyDrive/Colab Notebooks/Projeto_dissertacao/APP/DADOS_06_10 _1.CSV'

print("--- Primeiras 5 linhas do arquivo (Codificação Latin-1) ---")
try:
    with open(CAMINHO_DO_SEU_ARQUIVO, 'r', encoding='latin-1') as f:
        for i in range(5):
            print(f.readline().strip())
except Exception as e:
    print(f"Erro ao ler o arquivo: {e}")

print("\n--- Primeiras 5 linhas do arquivo (Codificação UTF-8) ---")
try:
    with open(CAMINHO_DO_SEU_ARQUIVO, 'r', encoding='utf-8') as f:
        for i in range(5):
            print(f.readline().strip())
except Exception as e:
    print(f"Erro ao ler o arquivo: {e}")
```

```
--- Primeiras 5 linhas do arquivo (Codificação Latin-1) ---
Leitura;Data;Hora;Temperatura(C);Umidade(%)
1;01/01/2000;00:00:20;33.1;38.2
2;01/01/2000;00:00:40;33;37.6
3;01/01/2000;00:01:00;32.9;37.5
4;01/01/2000;00:01:20;32.9;37.6

--- Primeiras 5 linhas do arquivo (Codificação UTF-8) ---
Leitura;Data;Hora;Temperatura(C);Umidade(%)
1;01/01/2000;00:00:20;33.1;38.2
2;01/01/2000;00:00:40;33;37.6
3;01/01/2000;00:01:00;32.9;37.5
4;01/01/2000;00:01:20;32.9;37.6
```

```
import pandas as pd
import os
from google.colab import drive
import numpy as np
import re

# ====================================================================
# 1. Montagem, Caminhos e Limpeza
# ====================================================================

print("Montando Google Drive...")
```

```
if not os.path.exists('/content/drive'):
    pass
else:
    print("Drive já montado.")

PASTA_DRIVE = '/content/drive/MyDrive/Colab Notebooks/Projeto_dissertacao'

# >>> CORREÇÃO DO CAMINHO: Adicionando a subpasta 'APP'
PASTA_ENTRADA = os.path.join(PASTA_DRIVE, 'APP')


NOME_ARQUIVO_ENTRADA = input("Digite o nome do arquivo TXT ou CSV para análise (Ex: PicoW.txt, S1.csv): ").strip()

if '.' not in NOME_ARQUIVO_ENTRADA:
    NOME_ARQUIVO_ENTRADA += '.txt'

ARQUIVO_ENTRADA = NOME_ARQUIVO_ENTRADA
CAMINHO_SAVE_ESTRUTURADO = os.path.join(PASTA_DRIVE, 'dados_estruturados.csv')

# >>> CORREÇÃO DO CAMINHO: O arquivo de entrada agora está na subpasta APP
CAMINHO_COMPLETO_ENTRADA = os.path.join(PASTA_ENTRADA, ARQUIVO_ENTRADA)

NOME_LOG = ARQUIVO_ENTRADA.rsplit('.', 1)[0].upper()

print(f"Caminho do arquivo de entrada definido: {CAMINHO_COMPLETO_ENTRADA}")
print(f"Caminho de saída estruturado definido: {CAMINHO_SAVE_ESTRUTURADO}")

if os.path.exists(CAMINHO_SAVE_ESTRUTURADO):
    try:
        os.remove(CAMINHO_SAVE_ESTRUTURADO)
        print(f"✅ Arquivo de resultados anteriores removido: {os.path.basename(CAMINHO_SAVE_ESTRUTURADO)}")
    except Exception as e:
        print(f"AVISO: Não foi possível remover o arquivo de resultados anteriores. {e}")
print("-" * 50)


# ====================================================================
# 2. FUNÇÕES DE ESTRUTURAÇÃO
# ====================================================================

def processar_dataframe(df, nome_log, caminho_saida):
    """Função auxiliar para limpeza e salvamento final do DataFrame."""
    if 'Dispositivo' not in df.columns:
        df['Dispositivo'] = nome_log

    df.dropna(subset=['Timestamp'], inplace=True)

    if df.empty:
        print(f"AVISO: O DataFrame de {nome_log} ficou vazio após a limpeza inicial (Timestamp/Dispositivo).")
        return False

    df['Dispositivo'].replace('', np.nan, inplace=True)
    df.dropna(subset=['Dispositivo'], inplace=True)

    if df.empty:
        print(f"AVISO: O DataFrame de {nome_log} ficou vazio após a limpeza final de Dispositivo.")
        return False

    df.set_index('Timestamp', inplace=True)
```

```
    df['Temperatura'] = pd.to_numeric(df['Temperatura'], errors='coerce')
    df['Umidade'] = pd.to_numeric(df['Umidade'], errors='coerce')
    df.dropna(subset=['Temperatura', 'Umidade'], inplace=True)

    if df.empty:
        print(f"AVISO: O DataFrame de {nome_log} ficou vazio após a limpeza de Temperatura/Umidade.")
        return False

    # Salvar
    if os.path.exists(caminho_saida):
        df[['Dispositivo', 'Temperatura', 'Umidade']].to_csv(caminho_saida, mode='a', header=False)
    else:
        df[['Dispositivo', 'Temperatura', 'Umidade']].to_csv(caminho_saida)

    print(f"Estruturação de {nome_log} concluída. Salvo em: {os.path.basename(caminho_saida)} (Total de {len(df)} linhas)")
    return True

def estruturar_log_regex(caminho_entrada, caminho_saida, nome_log):
    return False


def estruturar_log_csv(caminho_entrada, caminho_saida, nome_log):
    """
    Tenta estruturar focando no formato CSV/Tabular, com ajuste para o delimitador
    ponto e vírgula (;) e engine 'python' para maior compatibilidade.
    """
    print(f"\nTentando estruturação (CSV/Tabular) de: {os.path.basename(caminho_entrada)}")

    df = None
    COLUNAS_BRUTAS_IMAGEM1 = ['Contagem_Col', 'Data_Hora_Col', 'Temperatura_Col', 'Umidade_Col']

    # Prioridade para Excel (Ponto e Vírgula) e TAB
    separadores = [
        (';', 'CSV (;) - Corrigido'),
        ('\t', 'TAB'),
        (',', 'CSV (,)'),
        ('\\s+', 'Múltiplos Espaços/Regex')
    ]
    codificacoes = ['latin-1', 'utf-8'] # Mantém as duas codificações

    for sep_char, sep_nome in separadores:
        for encoding in codificacoes:
            try:
                # Usa 'python' engine para maior robustez, especialmente com delimitadores não-padrão
                engine_type = 'python'

                # === TENTATIVA 2: Leitura com Cabeçalho (Foco aqui!) ===
                df_teste = pd.read_csv(caminho_entrada, sep=sep_char,
                                       engine=engine_type, on_bad_lines='skip',
                                       encoding=encoding)

                # SOLUÇÃO CRÍTICA: Limpa espaços em branco nos nomes das colunas
                df_teste.columns = df_teste.columns.str.strip()

                colunas_presentes = set(df_teste.columns)

                # --- Lógica do NOVO FORMATO CSV (Data + Hora) ---
                data_hora_match = 'Data' in colunas_presentes and 'Hora' in colunas_presentes
```

```
            temp_match_novo = 'Temperatura(C)' in colunas_presentes
            umid_match_novo = 'Umidade(%)' in colunas_presentes

            if data_hora_match and temp_match_novo and umid_match_novo:
                df = df_teste.copy()
                print(f"INFO: Usando formato delimitado ({sep_nome}, {encoding}) com cabeçalho LIMPO (NOVO FORMATO CSV) para {nome_log}")

                df['Timestamp_Str'] = df['Data'].astype(str) + ' ' + df['Hora'].astype(str)

                df['Timestamp'] = pd.to_datetime(df['Timestamp_Str'], format='%d/%m/%Y %H:%M:%S', errors='coerce')

                if df['Timestamp'].isnull().all():
                     df['Timestamp'] = pd.to_datetime(df['Timestamp_Str'], format='%d/%m/%Y %H:%M', errors='coerce')

                df.rename(columns={
                    'Temperatura(C)': 'Temperatura',
                    'Umidade(%)': 'Umidade'
                }, inplace=True)

                if 'Dispositivo' not in df.columns:
                    df['Dispositivo'] = nome_log

                df = df[['Dispositivo', 'Timestamp', 'Temperatura', 'Umidade']].copy()

                if not df.dropna(subset=['Timestamp']).empty:
                    return processar_dataframe(df, nome_log, caminho_saida)

            # --- FIM da Lógica do NOVO FORMATO CSV (Padrão) ---


            # === TENTATIVA 1: Leitura Bruta (fallback) ===
            if sep_char not in ['\\s+']:
                df_bruto = pd.read_csv(caminho_entrada, sep=sep_char, header=None,
                                       names=COLUNAS_BRUTAS_IMAGEM1,
                                       encoding=encoding, engine='python', on_bad_lines='skip',
                                       skiprows=0)
                # ... (Lógica da Tentativa 1 omitida) ...

        except Exception as e:
            # print(f"Tentativa com {sep_nome} ({encoding}) falhou: {e}")
            continue

    print(f"AVISO TABULAR/CSV: O arquivo não pôde ser lido corretamente com nenhum delimitador/codificação. Verifique a estrutura do arquivo.")
    return False


# --- EXECUÇÃO DA ESTRUTURAÇÃO (TENTATIVA MÚLTIPLA) ---

if not estruturar_log_regex(CAMINHO_COMPLETO_ENTRADA, CAMINHO_SAVE_ESTRUTURADO, NOME_LOG):
    estruturar_log_csv(CAMINHO_COMPLETO_ENTRADA, CAMINHO_SAVE_ESTRUTURADO, NOME_LOG)


# ====================================================================
# 3. Carregamento Final
# ====================================================================

df_dados = pd.DataFrame()

try:
```

```
    df_dados = pd.read_csv(CAMINHO_SAVE_ESTRUTURADO, index_col=0, parse_dates=True)
    df_dados.rename(columns={'Temperatura': 'Temp', 'Umidade': 'Umid'}, inplace=True)
    df_dados['Temp'] = pd.to_numeric(df_dados['Temp'], errors='coerce')
    df_dados['Umid'] = pd.to_numeric(df_dados['Umid'], errors='coerce')
    df_dados.dropna(subset=['Temp', 'Umid'], inplace=True)

except FileNotFoundError:
    print("\nERRO FATAL: O arquivo estruturado não foi encontrado. A estruturação falhou.")
    exit()
except Exception as e:
    print(f"\nERRO: Falha ao carregar o arquivo estruturado. {e}")
    pass

if df_dados.empty:
    print(f"\nERRO FATAL: O DataFrame {NOME_LOG} está vazio. Verifique o formato do arquivo de entrada e os separadores usados.")
    exit()
else:
    print(f"\nDataFrame {NOME_LOG} carregado com sucesso. Iniciando Análise. (Total de {len(df_dados)} pontos)")

# ====================================================================
# 4. Cálculo das Médias ARITMÉTICAS
# ====================================================================

resultados_analise = []
grupos_dispositivo = df_dados.groupby('Dispositivo')

print("\n--- ⏳ Calculando Médias ARITMÉTICAS (Simples) ---")

for dispositivo, df_grupo in grupos_dispositivo:
    if len(df_grupo) < 1:
        continue

    media_temp = df_grupo['Temp'].mean()
    media_umid = df_grupo['Umid'].mean()

    resultados_analise.append({
        'Dispositivo': dispositivo,
        'Temp_Média_Aritmetica': media_temp,
        'Umid_Média_Aritmetica': media_umid,
        'Total_Pontos': len(df_grupo)
    })

df_resultados = pd.DataFrame(resultados_analise).set_index('Dispositivo')
df_resultados.dropna(inplace=True)


# ====================================================================
# 5. Apresentação dos Resultados
# ====================================================================

# 1. Cálculo da Média Aritmética GERAL
media_temp_geral = df_dados['Temp'].mean()
media_umid_geral = df_dados['Umid'].mean()

# 2. Apresentação do Resultado FINAL (Geral)
print("\n" + "="*90)
print(f"|   ANÁLISE FINAL: MÉDIA ARITMÉTICA GERAL do arquivo: {ARQUIVO_ENTRADA}      |")
print("="*90)
```

```
print("\n--- 📊 Média ARITMÉTICA por Dispositivo (Individual) ---")
print(df_resultados[['Temp_Média_Aritmetica', 'Umid_Média_Aritmetica', 'Total_Pontos']].to_markdown(floatfmt=".2f"))


print("\n--- 🌞 Média ARITMÉTICA GERAL do Arquivo ---")
print(f"│   Temperatura Média GERAL: {media_temp_geral:.2f}°C")
print(f"│   Umidade Média GERAL:     {media_umid_geral:.2f}%")
print("-----------------------------------------")
```

```
Montando Google Drive...
Drive já montado.
Digite o nome do arquivo TXT ou CSV para análise (Ex: PicoW.txt, S1.csv): DADOS_06_10_7.CSV
Caminho do arquivo de entrada definido: /content/drive/MyDrive/Colab Notebooks/Projeto_dissertacao/APP/DADOS_06_10_7.CSV
Caminho de saída estruturado definido: /content/drive/MyDrive/Colab Notebooks/Projeto_dissertacao/dados_estruturados.csv
✅ Arquivo de resultados anteriores removido: dados_estruturados.csv
--------------------------------------------------

Tentando estruturação (CSV/Tabular) de: DADOS_06_10_7.CSV
INFO: Usando formato delimitado (CSV (;) - Corrigido, latin-1) com cabeçalho LIMPO (NOVO FORMATO CSV) para DADOS_06_10_7
Estruturação de DADOS_06_10_7 concluída. Salvo em: dados_estruturados.csv (Total de 116 linhas)

DataFrame DADOS_06_10_7 carregado com sucesso. Iniciando Análise. (Total de 116 pontos)

--- ⏳ Calculando Médias ARITMÉTICAS (Simples) ---

==========================================================================================
│   ANÁLISE FINAL: MÉDIA ARITMÉTICA GERAL do arquivo: DADOS_06_10_7.CSV      │
==========================================================================================

--- 📊 Média ARITMÉTICA por Dispositivo (Individual) ---
| Dispositivo   |   Temp_Média_Aritmetica |   Umid_Média_Aritmetica |   Total_Pontos |
|:--------------|------------------------:|------------------------:|---------------:|
| DADOS_06_10_7 |                   33.13 |                   32.22 |         116.00 |

--- 🌞 Média ARITMÉTICA GERAL do Arquivo ---
│   Temperatura Média GERAL: 33.13°C
│   Umidade Média GERAL:     32.22%
-----------------------------------------
/tmp/ipython-input-654986235.py:63: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the


  df['Dispositivo'].replace('', np.nan, inplace=True)
```